

EA.2 Implementación de diagrama de clases

Tecnología en desarrollo de software.

ASIGNATURA:

Programación II: Orientada a objetos avanzada.

DOCENTE:

Boris Alberto Salleg Royero.

INTEGRANTES:

Valentina Stephany Kassar Acuña

Lina Marcela Muñoz

Maria Ines Julio Navarro

(GRUPO Actividad 2 5)

Septiembre 2026.

ÍNDICE.

1. Introducción.
2. Objetivos.
 - 2.1 Objetivos generales.
 - 2.2 Objetivos específicos.
3. Desarrollo de la AE.
 - 3.1 Descripción del proceso de implementación.
 - 3.2 Explicación de cómo se aplicaron los principios de la POO y los principios SOLID.
4. Conclusiones y reflexión personal.
 - 4.1 Conclusiones (acorde a los objetivos).
 - 4.2 Reflexión sobre lo aprendido y aporte a la práctica profesional.
5. Enlaces.
 - 5.1 Enlace repositorio github.
 - 5.2 Enlace video explicativo.

INTRODUCCIÓN.

En esta actividad se implementa en lenguaje Java el diagrama de clases UML diseñado en la Actividad 1, correspondiente al sistema de biblioteca; el propósito es aplicar los pilares de la Programación Orientada a Objetos (POO) y los principios SOLID.

En la tecnología en desarrollo de software o en cualquier rama de ingeniería de software, pasar de la teoría y el diseño a la práctica es un paso clave. En este trabajo tomamos el diagrama de clases UML que diseñamos en la actividad 1 para el sistema de gestión de biblioteca y lo llevamos a transformar en un proyecto real y funcional escrito en Java. El objetivo principal fue llevar todo lo que logramos diagramar previamente en el diagrama (Clases, Atributos, Métodos y Relaciones) hacia el código, logrando aplicar las mejores prácticas de programación.

A lo largo del proyecto o del trabajo, aplicamos directamente los pilares de la programación orientada a objetos (POO): usamos la abstracción para estructurar el sistema, el encapsulamiento para así proteger los datos de cada objeto, la herencia para no repetir código y el polimorfismo para hacer el programa más flexible. Además, incorporamos principios SOLID para garantizar que la arquitectura quede limpia y fácil de mantener. Todo el trabajo fue desarrollado en equipo utilizando Github y otras herramientas, lo cual nos permitió de manera colaborativa y registrar el aporte individual de cada integrante a través de commits y otras herramientas que nos permitieron trabajar de esta manera.

OBJETIVOS.

2.1 Objetivos generales.

- Implementar en lenguaje Java el diagrama de clases UML que diseñamos en la actividad 1, aplicando los pilares de POO, principios SOLID y un flujo de trabajo colaborativo en GitHub.

2.2 Objetivos específicos.

- Traducir de manera exacta las clases, interfaces, métodos y relaciones (herencia, composición, agregación y composición) del UML a código Java.
- Aplicar los pilares de POO en el código, demostrando el uso de sobrecarga, sobrescritura de métodos (`@Override`) y colecciones polimórficas.
- Aplicar al menos tres principios SOLID en la estructura del proyecto para lograr así un código modular, organizado y fácil de escalar.
- Coordinar el desarrollo de GitHub con commits y grabar un video de sustentación donde se evidencie la explicación del trabajo realizado.

DESARROLLO DE EA.

3.1 Descripción del proceso de implementación.

Tomando como guía el diagrama de la actividad 1, organizamos la solución en Java de la siguiente manera:

- **Interfaz base (CatalogoItem).** La creamos para definir las operaciones básicas que cualquier elemento del catálogo debe tener (getTitulo(), getDisponibilidad(), mostrarDetalle()).
- **Jerarquía de libros y herencia.** La clase libro implementa la interfaz CatalogoItem. Luego creamos la clase LibroDigital que extiende de libro mediante herencia, agregando sus atributos específicos (urlDescarga, formatoArchivo, tamanoMB) y sobrescribiendo el método mostrarDetalle().
- **Manejo de personas.** Diseñamos la clase abstracta Personas con los datos generales (id, nombre, email). De ella heredan la clase Autor (que guarda una lista agregada de los libros que ha escrito) y la clase Usuario (Que maneja los préstamos solicitados).
- **Gestión de préstamos y estados.** La clase Préstamo une a un Usuario con un Libro. Además utiliza EstadoPrestamo (ACTIVO, DEVUELTO, VENCIDO) para controlar así la situación de cada préstamo y calcular días de retraso.
- **Clase administradora (Biblioteca).** Este funciona como el núcleo del sistema. Contiene las listas (List<>) de libros, usuarios y préstamos, y se encarga de realizar la lógica principal: registrar préstamos, procesar devoluciones, y hacer búsquedas por título o autor.

3.2 Explicación de cómo se aplicaron los principios de la POO y los principios SOLID.

- **pilares de la POO:**
 - **Encapsulamiento.** Definimos todos los atributos como Private en cada clase. El acceso a los datos se hace únicamente a través de métodos *getters* y *setters*, protegiendo la información y validando reglas de negocio.
 - **Herencia.** La reutilizamos para al hacer que LibroDigital herede de Libro, y que tanto Usuario como Autor extiendan de la clase abstracta Persona.
 - **Polimorfismo.** Se observa en la lista List<Libro> dentro de Biblioteca. Esta lista almacena tanto objetos de tipo Libro como de LibroDigital, y al llamar a

mostrarDetalle(), Java ejecuta automáticamente la versión que corresponde a cada objeto.

- **Sobrecarga y Sobrescritura.** Aplicamos sobrescritura (`@Override`) en `LibroDogital.mostrarDetalle()` y `Usuario.mostrarInfo()`. También usamos sobrecarga en la clase `Biblioteca` para los métodos de búsqueda, permitiendo buscar libros por título o autor usando el mismo nombre de método.
- **Principios SOLID:**
 - **responsabilidad única (SRP).** Cada clase tiene una tarea bien definida. Por ejemplo, `PRÉSTAMO` solo administra las fechas y retrasos de una transacción, mientras que una `BIBLIOTECA` se encarga de la gestión general del catálogo.
 - **Abierto/cerrado (OCP).** Gracias a la interfaz `CatalogoItem`, si el día de mañana queremos agregar revistas o audiolibros, podemos hacerlo creando una nueva clase sin tener la necesidad de modificar la lógica existente en la biblioteca.
 - **Sustitución de Liskov (LSP).** `LibroDigital` es una subclase de `Libro` y se puede usar en cualquier parte del código donde se espere un libro sin romper el programa y sin causar errores inesperados.

CONCLUSIONES Y REFLEXIÓN PERSONAL.

4.1 Conclusiones.

- Logramos llevar con éxito el diseño UML a un entorno de programación real en Java, respetando las relaciones de composición, agregación y herencia que habíamos definido en la actividad anterior.
- Entendimos la importancia práctica de los pilares de la POO y los principios SOLID, los cuales nos permitieron escribir un código ordenado, claro y fácil de extender sin generar enredos.
- La experiencia de trabajar en GitHub fue indispensable para coordinar el proyecto en equipo, ya que este nos obligó a mantener un historial limpio y transparente de los aportes de cada una de las integrantes del grupo.

4.2 Reflexión sobre lo aprendido y aporte a la práctica profesional.

Como grupo, esta actividad nos sirvió para entender el valor real de planear y diseñar un sistema antes de ponernos a tirar o generar código. Ver como las clases se conectan y como la lógica del mundo real se traduce a objetos nos dió una visión mucho más profesional sobre el desarrollo de software. Nos dimos cuenta de que aplicar principios de diseño como SOLID no es solo teoría, sino una herramienta real para que el código no se vuelva inmanejable a medida que el proyecto crece. Además, trabajar con control de versiones en GitHub nos dejó una experiencia muy cercana a cómo se trabaja día a día en la industria del software.

ENLACES

5.1 Enlace repositorio GitHub.

https://github.com/DesarrolloMarce25/EA2-Implementacion_de_Diagrama_de_clases

5.2 Enlace video.

https://drive.google.com/file/d/1Zte9n_cpq9v-f9l3x9WT13D0QY8LMth_/view?usp=sharing